



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

INGENIERÍA EN COMPUTACIÓN

Sistemas Distribuidos

Reporte Proyecto 1 – Caso de estudio: Interfaz de paso de mensajes y Cluster HPC

NOMBRE DE LOS INTEGRANTES:

Andrea Saldaña Navarrete

María Fernanda Ayala Hernández

Hansel Yael Tepal Briseño

Luis Antonio Ugartechea González

PROFESORA: ELBA KAREN SÁENZ GARCÍA

GRUPO: 03

SEMESTRE: 2027-1

FECHA DE ENTREGA: 16 de marzo de 2026

Objetivo

Construir un sistema distribuido de cómputo de alto rendimiento (HPC) y ejecutar programas paralelos que distribuyan el trabajo entre los nodos que lo conforman.

Desarrollo

El desarrollo del proyecto se dividió en dos etapas principales: la construcción de un mini clúster HPC y la resolución de problemas utilizando el estándar MPI.

Parte I: Construcción del mini clúster HPC

Se construyó un mini clúster de cómputo de alto rendimiento utilizando máquinas virtuales y la distribución PelicanHPC, conformado por un nodo servidor y al menos dos nodos cliente. A continuación se describe el proceso seguido para su puesta en marcha.

Parte 1.1: Pasos de construcción

Paso 1: Creación de las máquinas virtuales. Se crean tres máquinas virtuales, todas configuradas con la distribución PelicanHPC: una actuará como *servidor* del clúster y las otras dos como *nodos cliente*.

Paso 2: Configuración de red y arranque del servidor. La máquina servidor se configura con **dos tarjetas de red**: una conectada a Internet, y otra configurada como una red aislada local (la *red cluster*) que es la que usará para comunicarse con los nodos cliente. A diferencia de los clientes, el servidor debe arrancar (*bootear*) desde un disco SATA que contiene cargada la imagen ISO de PelicanHPC.

Paso 3: Configuración de red y arranque de los clientes. Cada nodo cliente se configura con **una sola tarjeta de red**, conectada únicamente a la red aislada (*red cluster*) compartida con el servidor. A diferencia del servidor, los clientes deben arrancar por **red (PXE boot)** y no desde SATA ni desde una unidad de arranque local, ya que de esa forma no logran unirse correctamente al clúster.

Paso 4: Orden de encendido. Primero se encienden los dos nodos cliente, y hasta entonces se arranca (bootea) el servidor. Al iniciar el servidor es posible cargar el ambiente gráfico para tener una mejor referencia visual de lo que está ocurriendo, aunque este paso no es estrictamente necesario para el funcionamiento del clúster.

Paso 5: Inicialización del clúster. Ya con el servidor levantado, se ejecuta el comando de `pelican_setup` para inicializar el proceso de instalación/configuración del clúster HPC. Este comando habilita el multiprocesamiento en el servidor y comienza a detectar automáticamente los nodos

disponibles en la red aislada (los dos clientes previamente encendidos). Una vez que ambos nodos son detectados, el clúster queda levantado y listo para recibir trabajo distribuido.

Parte 1.2: Ejemplo de validación del clúster

Para validar el funcionamiento del clúster se ejecutó un programa en el que el proceso 0 envía un arreglo de 5 elementos de tipo flotante (con valores distintos) a otros cinco procesos. Cada proceso receptor imprime en pantalla el arreglo recibido, el nombre de la máquina/nodo en el que se ejecuta (obtenido con `MPI_Get_processor_name`) y su identificador de proceso (*rank*), demostrando así que el trabajo se distribuye realmente entre los distintos nodos del clúster. Se utilizó un mínimo de 6 procesos en total para cumplir con lo solicitado.

Listing 1: Fragmento clave: envío del proceso 0 a los demás y recepción con identificación de nodo

```
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &id);
MPI_Comm_size(MPI_COMM_WORLD, &np);

if (id == 0) {
    // ... se llena data[5] con 5 valores distintos ...
    for (int dest = 1; dest < np; dest++) {
        MPI_Send(data, 5, MPI_FLOAT, dest, 0, MPI_COMM_WORLD);
    }
} else {
    MPI_Recv(data, 5, MPI_FLOAT, 0, 0, MPI_COMM_WORLD, &status);

    char processor_name[MPI_MAX_PROCESSOR_NAME];
    int name_len;
    MPI_Get_processor_name(processor_name, &name_len);
    // ... se imprime id, processor_name y data[] ...
}

MPI_Finalize();
```

Funciones de MPI utilizadas y para qué sirven en este programa

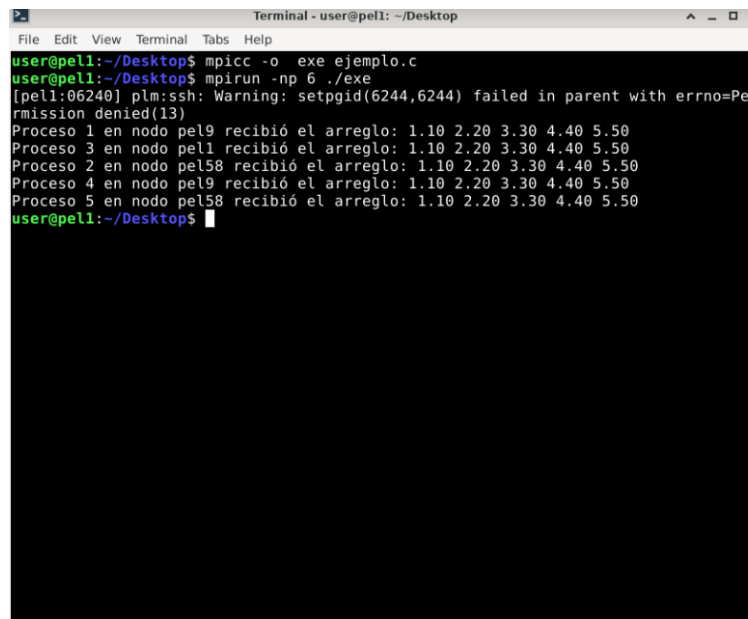
MPI_Comm_rank Obtiene el identificador (*rank*) del proceso que la llama. Se usa para decidir si el proceso actúa como emisor (`id == 0`) o como receptor, y para imprimirlo junto con el resultado recibido.

MPI_Comm_size Obtiene el número total de procesos (*np*). Se usa para que el proceso 0 sepa a cuántos procesos debe enviar el arreglo (`for dest = 1..np-1`).

MPI_Send Envía el arreglo data de forma bloqueante del proceso 0 a cada uno de los demás procesos. Se eligió comunicación punto a punto porque el mismo dato debe llegar, explícitamente, a cada proceso por separado.

MPI_Recv Recibe el arreglo enviado por el proceso 0. Es bloqueante, por lo que cada proceso receptor espera a tener el arreglo completo antes de imprimir el resultado.

MPI_Get_processor_name Obtiene el nombre de la máquina física/virtual (nodo) en la que corre el proceso. Es la función clave para *demostrar* que el trabajo se distribuyó entre distintos nodos del clúster y no solo entre procesos de una misma máquina.



```
Terminal - user@pel1: ~/Desktop
File Edit View Terminal Tabs Help
user@pel1:~/Desktop$ mpicc -o exe ejemplo.c
user@pel1:~/Desktop$ mpirun -np 6 ./exe
[pel1:06240] plm:ssh: Warning: setpgid(6244,6244) failed in parent with errno=Permission denied(13)
Proceso 1 en nodo pel9 recibió el arreglo: 1.10 2.20 3.30 4.40 5.50
Proceso 3 en nodo pel1 recibió el arreglo: 1.10 2.20 3.30 4.40 5.50
Proceso 2 en nodo pel58 recibió el arreglo: 1.10 2.20 3.30 4.40 5.50
Proceso 4 en nodo pel9 recibió el arreglo: 1.10 2.20 3.30 4.40 5.50
Proceso 5 en nodo pel58 recibió el arreglo: 1.10 2.20 3.30 4.40 5.50
user@pel1:~/Desktop$
```

Figura 1: Ejecución del programa de arreglo de 5 elementos

Parte II: Resolución de problemas utilizando MPI

Parte 2.1: Paso de arreglos entre procesos (anillo)

Se implementó un programa que realiza el paso de dos arreglos –uno de enteros (A) y otro de flotantes (B)– entre np procesos organizados en forma de anillo: el proceso 0 envía al proceso 1, este al proceso 2, y así sucesivamente hasta que el último proceso envía de regreso al proceso 0, cerrando el ciclo. Cada proceso que recibe los arreglos agrega un elemento nuevo a cada uno antes de reenviarlos, por lo que el tamaño de los arreglos crece en uno por cada nodo que los procesa. Se utilizaron etiquetas (*tags*) distintas para A y B para que **MPI_Recv** pueda distinguir ambos mensajes.

Listing 2: Fragmento clave: envío inicial en anillo (proceso 0), reenvío intermedio y recepción del cierre del ciclo

```
// --- Proceso 0: arranca el anillo y espera a que regrese ---
```

```

if (id == 0) {
    // ... A y B se inicializan con NELEMENTS valores ...
    MPI_Send(A, NELEMENTS, MPI_INT, 1, 0, MPI_COMM_WORLD);
    MPI_Send(B, NELEMENTS, MPI_FLOAT, 1, 1, MPI_COMM_WORLD);

    // ... realloc para el tamaño final: NELEMENTS + (np - 1) ...
    MPI_Recv(A, final_size, MPI_INT, np - 1, 0, MPI_COMM_WORLD, &status);
    MPI_Recv(B, final_size, MPI_FLOAT, np - 1, 1, MPI_COMM_WORLD, &status);
}
// --- Procesos 1..np-1: reciben, agregan un elemento y reenvían ---
else {
    MPI_Recv(A, recv_size, MPI_INT, id - 1, 0, MPI_COMM_WORLD, &status);
    MPI_Recv(B, recv_size, MPI_FLOAT, id - 1, 1, MPI_COMM_WORLD, &status);

    A[recv_size] = id; // se agrega un elemento nuevo
    B[recv_size] = (float)id + 0.5;

    int next_process = (id + 1) % np; // cierra el anillo al llegar a np-1
    MPI_Send(A, send_size, MPI_INT, next_process, 0, MPI_COMM_WORLD);
    MPI_Send(B, send_size, MPI_FLOAT, next_process, 1, MPI_COMM_WORLD);
}

```

Funciones de MPI utilizadas y para qué sirven en este programa

MPI_Send Envía los arreglos A y B al siguiente proceso del anillo. Se usa comunicación punto a punto (y no colectiva) porque el patrón de comunicación es estrictamente secuencial: cada proceso solo se comunica con su vecino inmediato.

MPI_Recv Recibe los arreglos del proceso anterior de forma bloqueante, garantizando que un proceso no agregue su elemento ni reenvíe hasta tener los datos completos del anterior.

MPI_Status Estructura que recibe información sobre el mensaje recibido en cada MPI_Recv (por ejemplo, el emisor real). Es obligatoria como parámetro de MPI_Recv.

```
Terminal - user@pell: ~/Desktop
File Edit View Terminal Tabs Help
user@pell:~/Desktop$ mpicc -o exe pasoMatrices.c
user@pell:~/Desktop$ mpirun -np 6 ./exe
Proceso 0 en nodo pell: Arreglo A inicial: 33 22 42 14 39
Proceso 0 en nodo pell: Arreglo B inicial: 1.8 1.6 4.8 0.0 1.6

Proceso 1 en nodo pel9: Arreglo A modificado: 33 22 42 14 39 1
Proceso 1 en nodo pel9: Arreglo B modificado: 1.8 1.6 4.8 0.0 1.6 1.5

Proceso 2 en nodo pel58: Arreglo A modificado: 33 22 42 14 39 1 2
Proceso 2 en nodo pel58: Arreglo B modificado: 1.8 1.6 4.8 0.0 1.6 1.5 2.5

Proceso 3 en nodo pell: Arreglo A modificado: 33 22 42 14 39 1 2 3
Proceso 3 en nodo pell: Arreglo B modificado: 1.8 1.6 4.8 0.0 1.6 1.5 2.5 3.5

Proceso 4 en nodo pel9: Arreglo A modificado: 33 22 42 14 39 1 2 3 4
Proceso 4 en nodo pel9: Arreglo B modificado: 1.8 1.6 4.8 0.0 1.6 1.5 2.5 3.5 4

Proceso 5 en nodo pel58: Arreglo A modificado: 33 22 42 14 39 1 2 3 4 5
Proceso 5 en nodo pel58: Arreglo B modificado: 1.8 1.6 4.8 0.0 1.6 1.5 2.5 3.5 4.5 5.5

-- RESULTADOS FINALES AL REGRESAR AL PROCESO 0 --
Proceso 0 en nodo pell: Arreglo A final: 33 22 42 14 39 1 2 3 4 5
Proceso 0 en nodo pell: Arreglo B final: 1.8 1.6 4.8 0.0 1.6 1.5 2.5 3.5 4.5 5.5

user@pell:~/Desktop$
```

Figura 2: Ejecución del programa de paso de arreglos en anillo

Parte 2.2: Renglón con la suma mínima de una matriz

Dada una matriz A de tamaño $N \times M$ con números enteros, se desarrolló un programa distribuido en MPI que determina qué renglón de la matriz tiene la menor suma de sus elementos. El proceso 0 genera la matriz completa y la reparte entre los procesos —un renglón por proceso— utilizando la función colectiva `MPI_Scatter`. Cada proceso calcula la suma de los elementos de su propio renglón y la reporta de vuelta al proceso 0 mediante `MPI_Gather`, quien determina el renglón con la suma mínima. Se supone que N es divisible entre el número de procesos y se usa un mínimo de 4 procesos.

Listing 3: Fragmento clave: reparto de renglones y recolección de sumas con funciones colectivas

```
MPI_Comm_rank(MPI_COMM_WORLD, &id);
MPI_Comm_size(MPI_COMM_WORLD, &np);
int N = np; // un renglon por proceso

// ... el proceso 0 genera A (N x M) e imprime la matriz inicial ...

MPI_Scatter(A, M, MPI_INT, local_row, M, MPI_INT, 0, MPI_COMM_WORLD);

int local_sum = 0;
for (int i = 0; i < M; i++) local_sum += local_row[i];

MPI_Gather(&local_sum, 1, MPI_INT, all_sums, 1, MPI_INT, 0, MPI_COMM_WORLD);

if (id == 0) {
```

```

// ... se recorre all_sums[] para encontrar el valor minimo y su indice
// (el indice = numero de renglon, porque cada proceso hizo 1 renglon) ...
}

```

Funciones de MPI utilizadas y para qué sirven en este programa

MPI_Comm_rank / MPI_Comm_size Dan el identificador de cada proceso y el total de procesos. Se usan para fijar $N = np$ (un renglón por proceso) y para que el proceso 0 recorra correctamente el arreglo `all_sums`.

MPI_Scatter Reparte la matriz A entre todos los procesos, entregando a cada uno M elementos (un renglón distinto) en `local_row`. Se usó esta función colectiva –y no varios `MPI_Send` individuales– porque el enunciado pide explícitamente distribuir la matriz con comunicaciones colectivas de MPI, y porque expresa de forma natural el patrón *uno-a-todos con datos distintos para cada quien*.

MPI_Gather Recolecta en el proceso 0 el valor `local_sum` calculado por cada proceso, guardándolos en orden de *rank* dentro de `all_sums`. Se usó porque el patrón es el inverso de Scatter: *todos-a-uno*, cada proceso aporta un único valor.

Se prefirió el par `MPI_Scatter/MPI_Gather` sobre comunicación punto a punto porque simplifica el código, evita tener que programar un ciclo de `MPI_Send/MPI_Recv` para cada proceso y reduce el riesgo de interbloqueos.

```

File Edit View Terminal Tabs Help
user@pell:~/Desktop$ mpicc -o exe renglonSuma.c
user@pell:~/Desktop$ mpirun -np 6 ./exe
--- Matriz Inicial A (6 x 6) ---
Renglón 0: 46 34 31 9 9 44
Renglón 1: 26 8 32 5 11 40
Renglón 2: 33 22 14 11 18 43
Renglón 3: 33 15 47 43 25 19
Renglón 4: 32 35 12 4 5 23
Renglón 5: 46 1 10 30 11 19

Proceso 0 en nodo pell calculó la suma de su renglón: 173
Proceso 3 en nodo pell calculó la suma de su renglón: 182
Proceso 2 en nodo pel58 calculó la suma de su renglón: 141
Proceso 1 en nodo pel9 calculó la suma de su renglón: 122
Proceso 5 en nodo pel58 calculó la suma de su renglón: 117
Proceso 4 en nodo pel9 calculó la suma de su renglón: 111

--- RESULTADO FINAL ---
El renglón con la suma menor es el renglón 4.
La suma de sus elementos es: 111
user@pell:~/Desktop$

```

Figura 3: Ejecución del programa de determinación del renglón con la suma mínima

Parte III: Participación de los integrantes

Integrante	Actividades realizadas
María Fernanda Ayala Hernández	Desarrollo del programa de paso de arreglos en anillo (Parte 2.1); redacción y formato del documento de reporte.
Andrea Saldaña Navarrete	Desarrollo del programa de determinación del renglón con la suma mínima usando Scatter/Gather (Parte 2.2); redacción y formato del documento de reporte.
Hansel Yael Tepal Briseño	Investigación y configuración de PelicanHPC; puesta en marcha del nodo servidor y los nodos cliente del clúster.
Luis Antonio Ugartechea González	Investigación y configuración de PelicanHPC; recopilación de capturas; grabación y edición del video de evidencia del clúster.

Cuadro 1: Participación de los integrantes del equipo.

Conclusiones

Con el desarrollo de este proyecto se comprobó, de forma práctica, cómo un mismo problema puede resolverse distribuyendo el trabajo entre distintos procesos y nodos físicos usando el estándar MPI. La construcción del clúster con PelicanHPC permitió observar de primera mano los retos reales de un entorno distribuido: la configuración de red entre nodos, la sincronización de los procesos y la necesidad de identificar en qué máquina se ejecuta cada uno.

Al implementar los dos programas se hizo evidente la diferencia entre la comunicación punto a punto (MPI_Send/MPI_Recv), útil cuando el patrón de comunicación es secuencial o específico entre un par de procesos –como en el paso de arreglos en anillo–, y la comunicación colectiva (MPI_Scatter/MPI_Gather), más adecuada cuando el mismo tipo de operación (repartir o recolectar datos) involucra a todos los procesos a la vez –como en el reparto de renglones de la matriz–. Usar la función colectiva correcta simplificó considerablemente el código y evitó tener que programar manualmente ciclos de envío y recepción.

En general, el proyecto cumplió el objetivo de construir un sistema HPC funcional y de resolver problemas reales de forma distribuida, reforzando los conceptos de paralelismo, comunicación entre procesos y balanceo de carga vistos en la clase de Sistemas Distribuidos.

Referencias

- Creel, M. (2008). *PelicanHPC Tutorial*. Universitat Autònoma de Barcelona.
<https://digital.csic.es/bitstream/10261/7566/1/74908.pdf>

- Karen Sáenza - Clases FI. *clusterHPC* .
https://www.youtube.com/watch?v=F_LFf0lhbRA
- Karen Sáenza - Clases FI. *clusterHPC con distribución Pelican-hpc* .
<https://www.youtube.com/watch?v=YYfAFJudSgk>
- Sáenz García, E. K. (2026). *Notas de clase: Sistemas Distribuidos – Tema 1 y Tema 2*. Facultad de Ingeniería, UNAM.

Video y Repositorio

- Repositorio Github
https://github.com/lethalSopaper/proyectos_sistemas_distribuidos/tree/e051eeaf537859d1986e01proyecto1
- Video Explicativo
<https://drive.google.com/file/d/1SvNguiUIT2hdQzMOZMrYAVmXpXVB0bag/view?usp=sharing>